

ITERATORS

Definition

An **iterator** is a value that allows you to **traverse a sequence** of items **one at a time**.

In Rust, any type that implements the **Iterator trait** can be used in a for loop or with iterator methods like `.map()`, `.filter()`, and `.collect()`, etc.

Understand the basics


```
.iter()  
    .filter()  
    .map()  
    .collect::<<>>()
```

Benefits

✓ Why Iterators Are Useful

Benefit	What It Means
Cleaner code	Avoids manual indexing (<code>for i in 0..n</code>) — just say <code>for item in collection</code>
Memory efficient	Values can be processed lazily (on demand), not all loaded at once
Chainable operations	Combine actions like <code>filter</code> , <code>map</code> , <code>sum</code> in a readable way
Safer iteration	No risk of out-of-bounds errors like with raw indexing
Works with all collections	Strings, arrays, vectors, hashmaps, file lines, etc.

In Rust, the fundamental piece of iterator functionality comes in the form of a trait:

```
pub trait Iterator {  
    type Item;  
    fn next(&mut self) → Option<Self::Item>;  
}
```

The full definition of this trait is extensive, spanning over 4,000 lines of code and documentation. However, the foundation of iterator functionality in Rust is built upon two key components:

<https://doc.rust-lang.org/std/iter/trait.Iterator.html>

**std**
1.83.0
(90b35a623 2024-11-26)

Iterator

Required
Associated Types

Item

 Required Methods

Provided Methods

advance_by

all

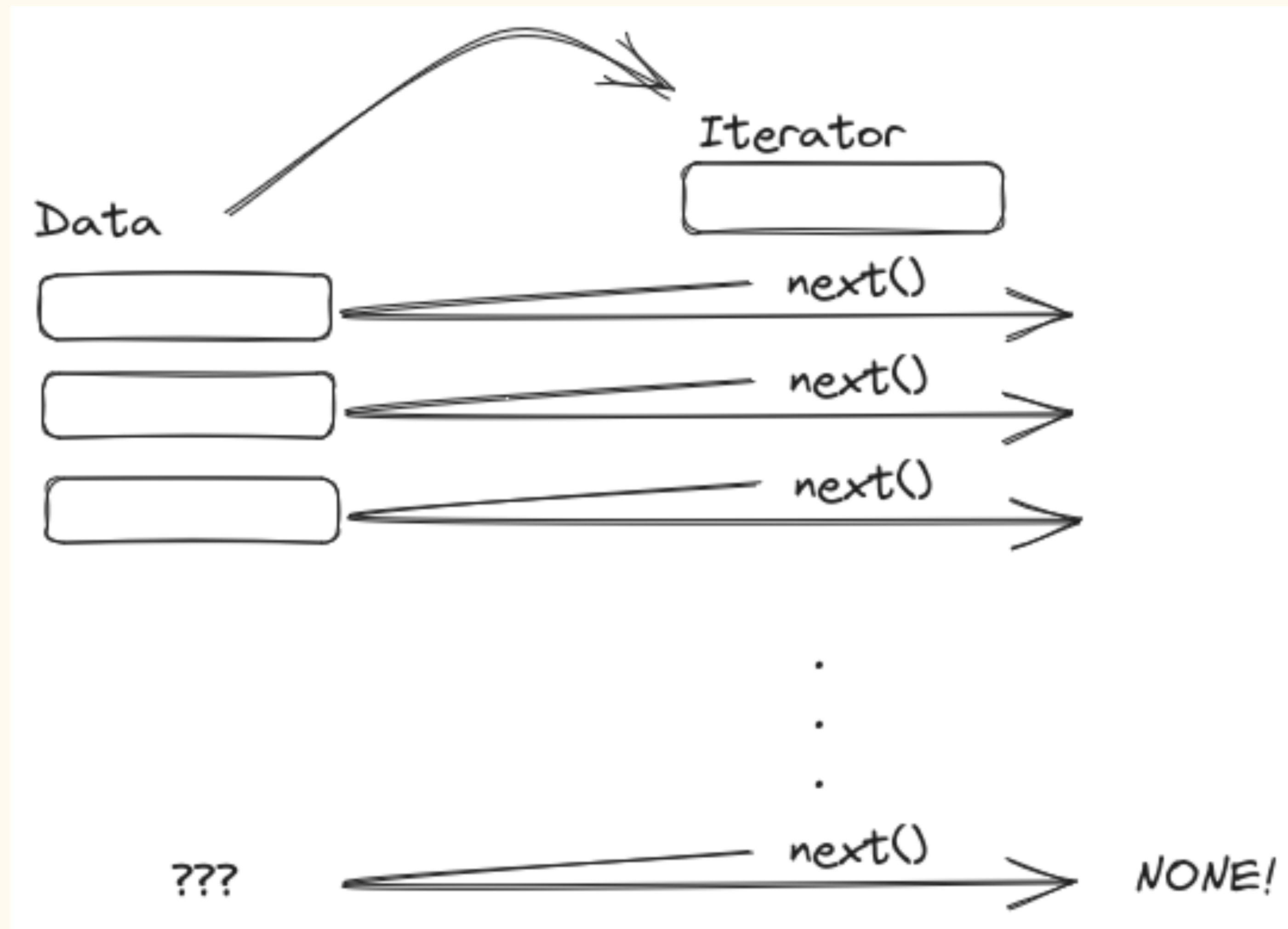
any

Type 'S' or '/' to search, '?' for more options...

std::iter
Trait Iterator 
Since 1.0.0 · [source](#)

```
pub trait Iterator {  
    type Item;  
  
    // Required method  
    fn next(&mut self) -> Option<Self::Item>;  
  
    // Provided methods  
    fn next_chunk<const N: usize>(  
        &mut self,  
    ) -> Result<[Self::Item; N], IntoIter<Self::Item, N>>  
        where Self: Sized { ... }  
  
    fn size_hint(&self) -> (usize, Option<usize>) { ... }  
}
```

Exercise 1: Iterator basics



From the std



```
let a = [1, 2, 3];
```

```
let mut iter = a.into_iter();
```

```
// A call to next() returns the next value...
```

```
assert_eq!(Some(1), iter.next());
```

```
assert_eq!(Some(2), iter.next());
```

```
assert_eq!(Some(3), iter.next());
```

```
// ... and then None once it's over.
```

```
assert_eq!(None, iter.next());
```

```
// More calls may or may not return `None`. Here, they always will.
```

```
assert_eq!(None, iter.next());
```

```
assert_eq!(None, iter.next());
```

Adapter vs Consumer Methods

Adapter vs Consumer

Adapter Methods	Consumer methods
Used to transform, filter, or modify an iterator. They are lazy , meaning they don't do any work until the iterator is consumed.	Execute the iterator and produce a final result. They are eager , meaning they consume the iterator immediately.
<i>map, filter, take, ...</i>	<i>collect, sum, for_each, ...</i>

Exercise 2: Adapter vs Consumer Methods

Consume does not mean take ownership

✅ Why They're Called Consumer Methods

They are called **consumer methods** because they **consume the iterator itself** — meaning:

- They **drive the iteration forward**
- They **exhaust** the iterator (you can't use it again)
- They **produce a final result** (e.g., `sum`, `collect`, `count`)

So, the term **"consumer"** refers to **consuming the iterator**, not necessarily the original collection it came from.

3 ways to create an iterator

✓ Three Common Ways to Get an Iterator

Method	What it Does	Ownership	Type of Items
<code>.iter()</code>	Iterates by shared reference	Doesn't move	<code>&T</code>
<code>.iter_mut()</code>	Iterates by mutable reference	Doesn't move	<code>&mut T</code>
<code>.into_iter()</code>	Iterates by moving values (ownership)	Moves the data	<code>T</code>

Example Comparison

rust

```
let vec = vec![10, 20, 30];
```

```
vec.iter();           // yields &i32
```

```
vec.iter_mut();       // yields &mut i32
```

```
vec.into_iter();      // yields i32 (takes ownership of vec)
```

Summary

- You **don't always need** to use `.into_iter()` explicitly — it's often called for you.
- Use `.iter()` or `.iter_mut()` when you want to borrow the collection.
- Use `.into_iter()` when you want to move (own) the data.

1. Adapter methods

Adapter Methods (return another iterator)

These are **lazy** — they don't consume values until iterated.

- `map(f)` – Applies a function to each item.
- `filter(predicate)` – Filters elements based on a predicate.
- `enumerate()` – Yields `(index, item)` tuples.
- `take(n)` – Yields the first `n` elements.
- `skip(n)` – Skips the first `n` elements.
- `chain(iter2)` – Joins two iterators.
- `zip(iter2)` – Pairs elements from two iterators.
- `flat_map(f)` – Maps and then flattens nested iterators.
- `peekable()` – Converts into an iterator that allows peeking at the next element.
- `rev()` – Reverses the order (requires `DoubleEndedIterator`).
- `cycle()` – Repeats the iterator infinitely.
- `inspect(f)` – Executes a function on each item for debugging, without modifying them.

2. Consumer methods

Consumer Methods (consume the iterator)

These methods **take ownership** of the iterator and produce a final result.

- `collect()` – Collects items into a collection (e.g., `Vec`, `HashSet`).
- `for_each(f)` – Runs a function on each element (like a `for` loop).
- `fold(init, f)` – Reduces elements into a single value (like `reduce`).
- `sum()`, `product()` – Computes the sum or product of items.
- `find(predicate)` – Finds the first element that matches.
- `any(predicate)`, `all(predicate)` – Checks conditions across elements.
- `count()` – Counts the number of elements.
- `last()` – Returns the last element.
- `nth(n)` – Returns the `nth` element (0-based).
- `partition(predicate)` – Splits elements into two collections.

Exercises on methods

Your turn!

Adapter Methods	Consumer Methods
<code>map</code> – Transforms each item using a function.	<code>collect</code> – Gathers items into a collection like <code>Vec</code> or <code>HashMap</code> .
<code>filter</code> – Keeps items that satisfy a predicate.	<code>for_each</code> – Runs a function on each item for side effects.
<code>enumerate</code> – Pairs each item with its index.	<code>fold</code> – Accumulates values with an initial seed and function.
<code>take</code> – Yields only the first <code>n</code> items.	<code>reduce</code> – Combines all items into one, starting with the first.
<code>skip</code> – Skips the first <code>n</code> items.	<code>sum</code> – Returns the sum of all items.
<code>chain</code> – Combines two iterators into one.	<code>product</code> – Returns the product of all items.
<code>zip</code> – Pairs items from two iterators into tuples.	<code>find</code> – Returns the first item that matches a condition.
<code>flat_map</code> – Maps and flattens nested iterators.	<code>position</code> – Returns the index of the first matching item.
<code>peekable</code> – Allows peeking at the next item without consuming it.	<code>any</code> – Checks if any item satisfies a condition.
<code>rev</code> – Reverses the iterator (if supported).	<code>all</code> – Checks if all items satisfy a condition.
<code>cycle</code> – Repeats the iterator endlessly.	<code>count</code> – Returns the number of items.
<code>inspect</code> – Runs a function on each item for debugging.	<code>last</code> – Returns the final item from the iterator.
<code>cloned</code> – Clones each item (useful for iterators over references).	<code>nth</code> – Returns the nth item (0-based).
<code>copied</code> – Copies each item (for types that implement <code>Copy</code>).	<code>max</code> – Returns the largest item.
<code>fuse</code> – Once exhausted, always returns <code>None</code> even if underlying iterator misbehaves.	<code>min</code> – Returns the smallest item.
	<code>partition</code> – Splits items into two groups based on a predicate.



■ ■ ■ ■ .map(|■| ●) → ● ● ● ●
■ ■ ● ■ .filter(■) → ■ ■ ■
■ ● ● ● .filter_map(|■| ●) → ●
● ● ■ ■ .find(■) → ■
● ● ■ ■ .any(■) → true
● ● ■ ■ .all(■) → false
■ ■ ● ■ .fold(■) → 3
■ ■ ● ■ .take(2) → ■ ■
■ ■ ● ■ .skip(2) → ● ■
● ■ ● ■ .partition(■) → ■ ■ | ● ●



Jan Cibulka

Rust Iterator Methods 🦀